

Classe abstrata, interfaces e polimorfismo

CST em Análise e Desenvolvimento de Sistemas

Prof. Emerson Ribeiro de Mello

mello@ifsc.edu.br

Licenciamento



Slides licenciados sob [Creative Commons "Atribuição 4.0 Internacional"](https://creativecommons.org/licenses/by/4.0/)

Classe abstrata

Jogo Java of Empires

Fortemente inspirado no *Age of Empires* da Microsoft – <https://www.ageofempires.com>



Fonte: Microsoft

■ Aldeão

- Vida: 25
- Ataque: 1
- Velocidade: 0.8



Fonte: Microsoft

■ Arqueiro

- Vida: 35
- Ataque: 2
- Velocidade: 1



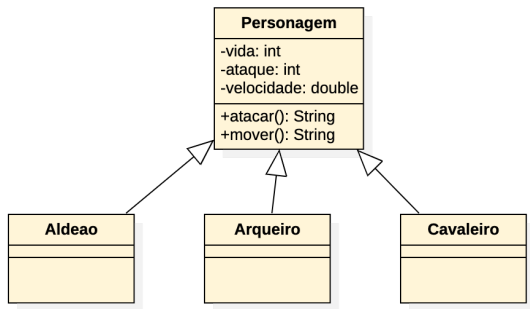
Fonte: Microsoft

■ Cavaleiro

- Vida: 50
- Ataque: 3
- Velocidade: 2

Jogo Java of Empires

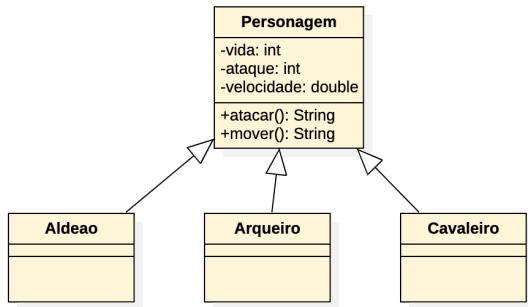
Proposta de modelagem 1



- O fato de existir um método `mover` na classe `Personagem` garante que todas as classes que herdam de `Personagem` possuam o método `mover`
- Porém, não obriga que o método `mover` seja sobrescrito nas subclasses
 - Todos teriam o mesmo comportamento de movimento, o que não é desejável

Jogo Java of Empires

Proposta de modelagem 1



- No jogo, só é possível jogar com Aldeao, Arqueiro e Cavaleiro
- Porém, a modelagem permite criar uma instância de **Personagem**, o que não faz sentido no contexto do jogo

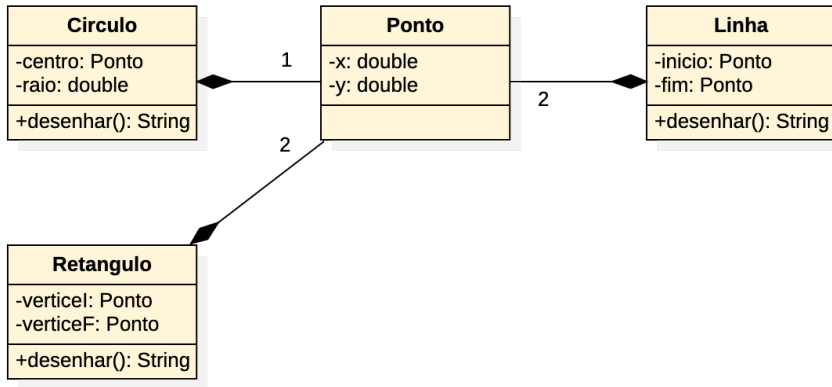
```
Personagem p = new Personagem();  
p.mover();
```

Aplicativo para desenho vetorial 2D (geometria plana)

- Linha
- Círculo
- Retângulo

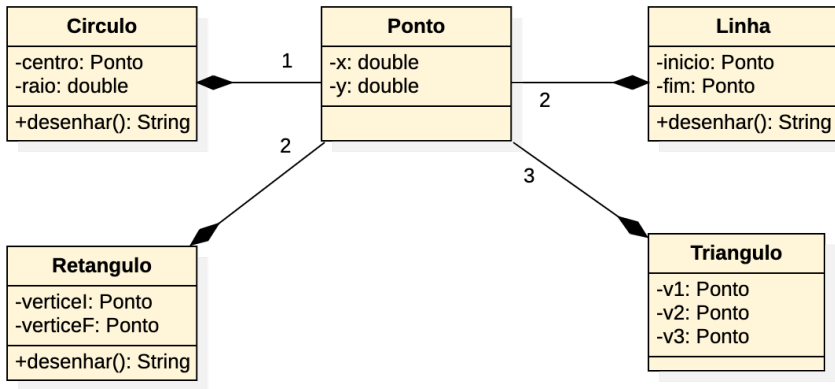
Aplicativo para desenho vetorial 2D (geometria plana)

- Linha
- Círculo
- Retângulo



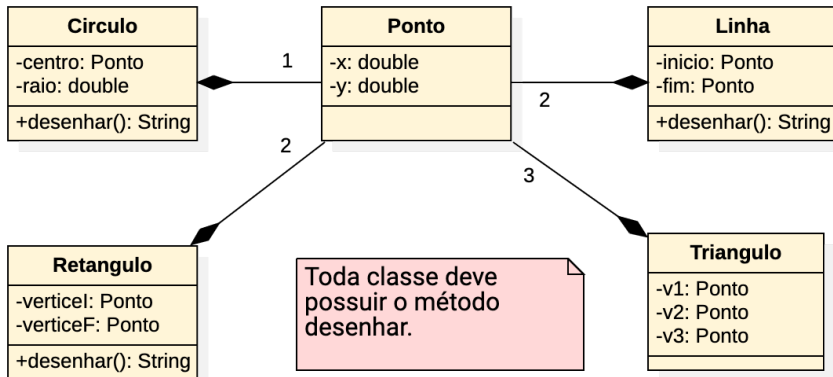
Aplicativo para desenho vetorial 2D (geometria plana)

- Linha
- Círculo
- Retângulo
- Triângulo



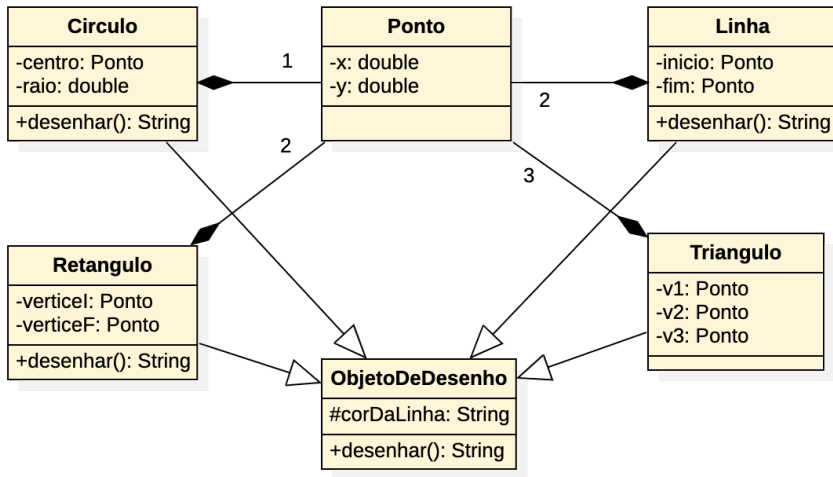
Aplicativo para desenho vetorial 2D (geometria plana)

- Linha
- Círculo
- Retângulo
- Triângulo



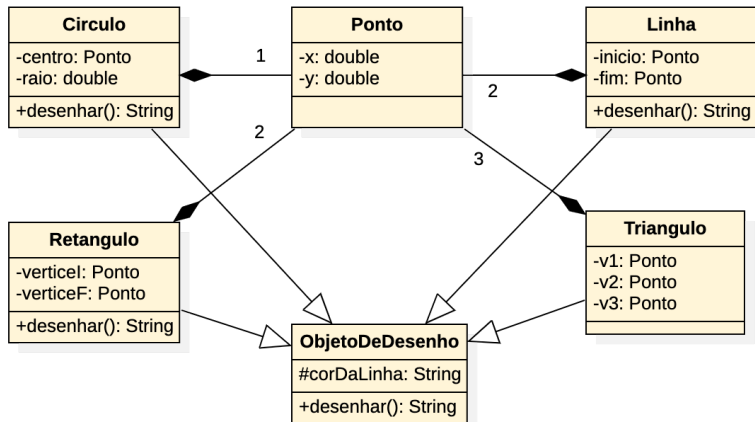
Aplicativo para desenho vetorial 2D (geometria plana)

- Linha
- Círculo
- Retângulo
- Triângulo



Aplicativo para desenho vetorial 2D (geometria plana)

- Linha
- Círculo
- Retângulo
- Triângulo



Toda classe deve possuir o método desenhar.
Esse modelo ainda não garante isso!

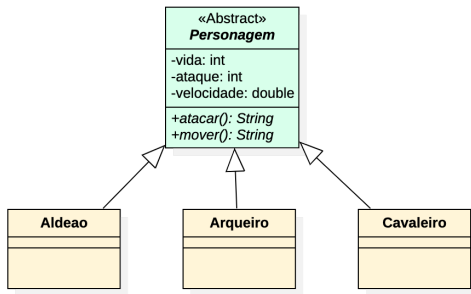
Classe abstrata em Java

- Uma **classe abstrata** pode conter atributos, métodos concretos e métodos abstratos, porém não pode ser instanciada
 - **Métodos concretos** possuem implementação
 - **Métodos abstratos** não possuem implementação
- Todo **método abstrato** deve ser **obrigatoriamente sobrescrito** pelas subclasses
 - Se não for sobrescrito, a subclasse deve ser abstrata
- **Métodos concretos** podem ser sobrescritos, mas não é obrigatório

Classe abstrata pode ser usada para definir uma **classe base** que será estendida por outras classes, obrigando que as subclasses implementem os métodos abstratos

Java of Empires

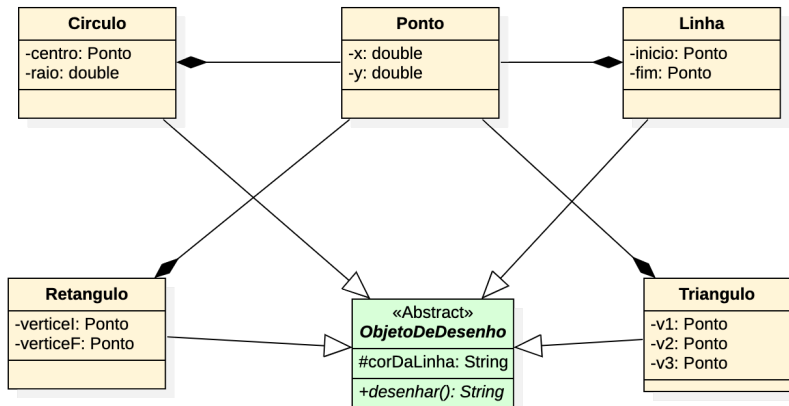
Proposta de modelagem 2



- Personagem é uma **classe abstrata**
 - Não pode ser instanciada
- Os métodos mover e atacar são **abstratos**
 - Obriga que as subclasses sobrescrevam esses métodos
- Cada objeto das classes concretas Aldeao, Arqueiro e Cavaleiro é um Personagem, e portanto, pode ser tratado como tal

```
Aldeao a = new Aldeao();
a.mover();
a.setVida(25);
```

Aplicativo para desenho vetorial 2D (geometria plana)



Esse modelo obriga, a toda classe que herdar de **ObjetoDeDesenho**, implementar o método abstrato `desenhar()`

Aplicativo para desenho vetorial 2D (geometria plana)

```
public abstract class ObjetoDeDesenho{  
    public abstract String desenhar();  
}
```

```
public class Circulo extends ObjetoDeDesenho{  
    private Ponto centro;  
    private double raio;  
  
    // Classe Circulo é obrigada a sobrescrever o método desenhar  
    @Override  
    public String desenhar(){  
        return String.format("Desenhando circulo com raio %.2f", raio);  
    }  
}
```


Interface

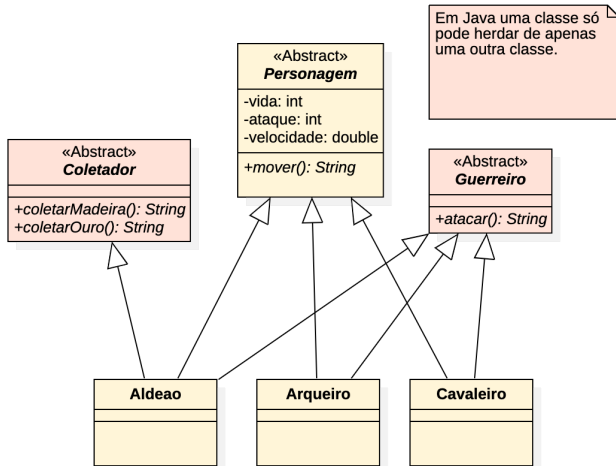
Herança múltipla

Em Java, interfaces permitem herança múltipla de tipos

- **Herança múltipla** é um conceito de orientação a objetos onde uma classe pode herdar atributos e métodos de mais de uma classe
 - A linguagem C++ possui o conceito de **herança múltipla**
- Em Java, uma classe pode herdar de apenas uma outra classe
 - Evita as complicações relacionadas a **herança múltipla de estados** – habilidade de herdar atributos de múltiplas classes
- Em Java, uma classe pode implementar várias **interfaces**, permitindo assim o conceito de **herança múltipla de tipos**
 - **herança múltipla de implementação** – habilidade de herdar as definições de métodos de múltiplas interfaces

Java of Empires

Modelagem com herança múltipla



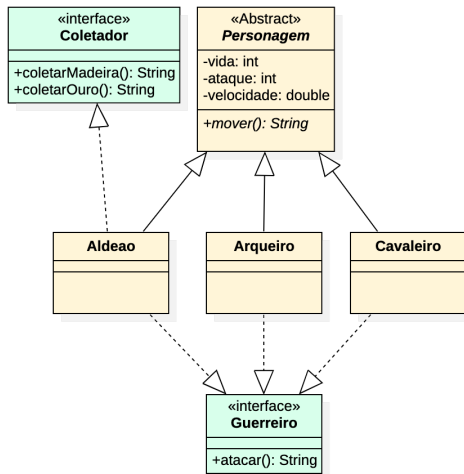
Interface

Uma Interface em Java é semelhante a uma classe abstrata, ou seja, é um tipo de dado que pode ser usado para definir um contrato que deve ser implementado por uma classe

- Todos os atributos são `public static final`
 - Atributos são constantes e pertencem à interface, não às classes que a implementam
- Todos os métodos são `public abstract` por padrão
- Métodos `static` e `default` podem possuir implementação

Java of Empires

Modelagem com interface



```
public interface Guerreiro{
    String atacar();
}

public class Aldeao extends Personagem
    implements Guerreiro, Coletador {

    @Override
    public String atacar(){
        return "Aldeao atacando";
    }

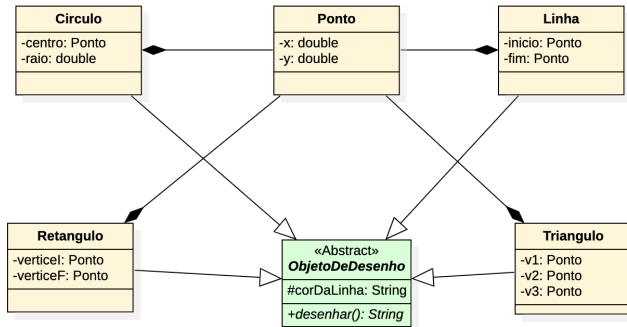
    @Override
    public String coletarMadeira(){
        return "Aldeao coletando madeira";
    }

    @Override
    public String coletarOuro(){
        return "Aldeao coletando ouro";
    }
}
```

Aplicativo para desenho vetorial 2D (geometria plana)

Quais ajustes devemos fazer na modelagem para que todas as formas geométricas tenham desenhar, calcular a área e o perímetro?

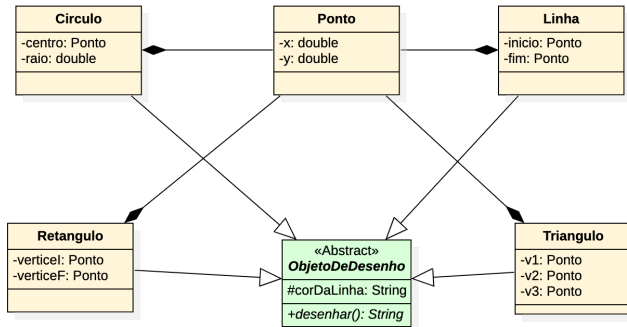
- 1 Adicionar métodos `calcularArea` e `calcularPerimetro` em `ObjetoDeDesenho`?
- 2 Criar uma nova classe abstrata `FormaGeometrica`?
- 3 Outra alternativa?



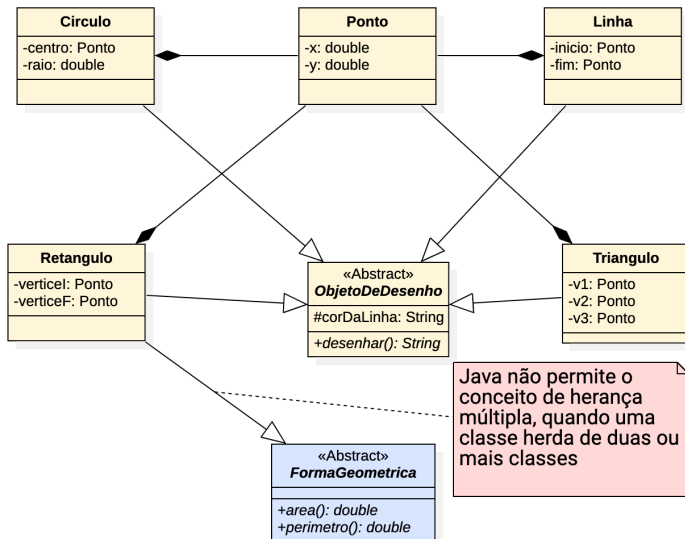
Aplicativo para desenho vetorial 2D (geometria plana)

Quais ajustes devemos fazer na modelagem para que todas as formas geométricas tenham desenhar, calcular a área e o perímetro?

- 1 Adicionar métodos `calcularArea` e `calcularPerimetro` em `ObjetoDeDesenho`?
- 2 Criar uma nova classe abstrata `FormaGeometrica`?
- 3 Outra alternativa?

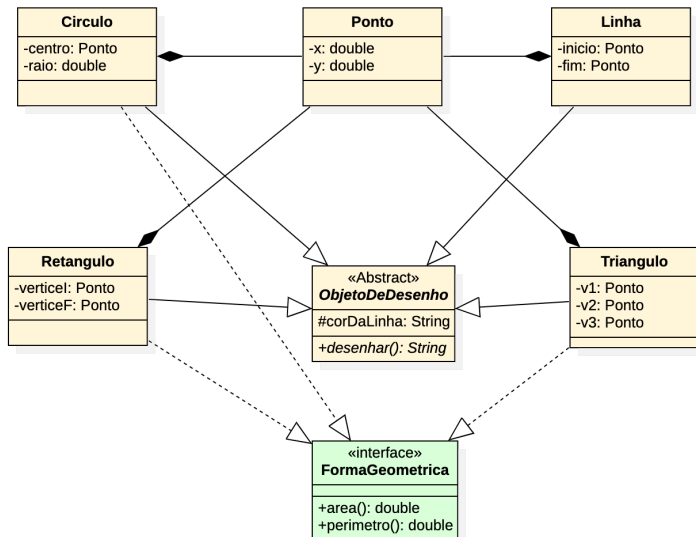


Aplicativo para desenho vetorial 2D (geometria plana)



Java não permite o conceito de herança múltipla, quando uma classe herda de duas ou mais classes

Aplicativo para desenho vetorial 2D (geometria plana)

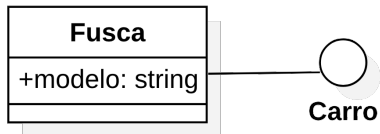
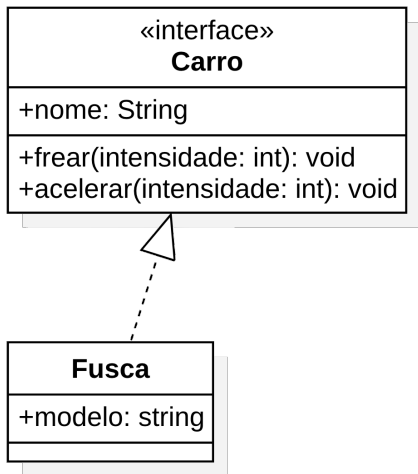


Exercício

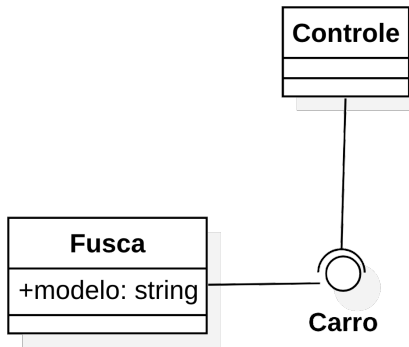
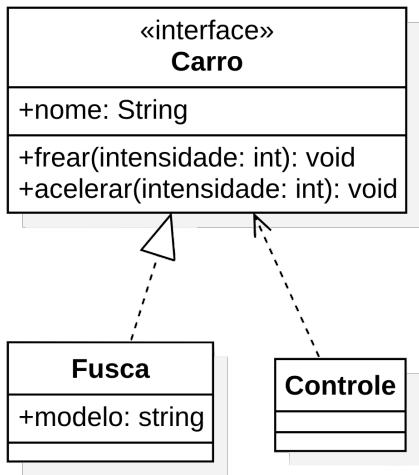
Jogo de corrida

- Um fabricante de jogo de corrida gostaria de **permitir que seu jogo fosse estendido por outras pessoas** de forma que **possam criar seus próprios carros**
- **Deve-se garantir que todos os carros possuam os mesmos métodos**
 - acelerar, frear, ligar e desligar
- Cada carro pode ter um comportamento diferente ao acelerar, frear, ligar e desligar
- Cada carro pode ser de uma marca diferente, ter uma cor diferente, um modelo diferente

Diferentes formas para representar interfaces na UML



Diferentes formas para representar interfaces na UML



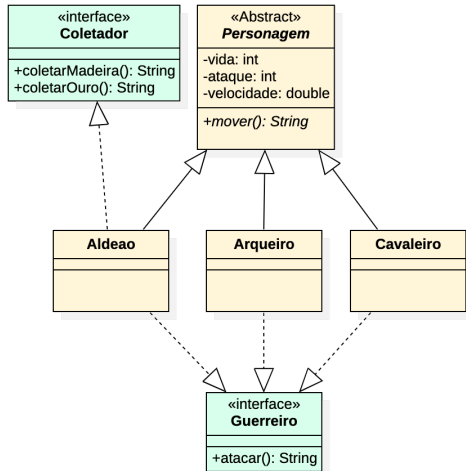
Polimorfismo

Polimorfismo

Significado: “muitas formas”

- É a capacidade de um objeto poder ser referenciado de várias formas
 - Permite que um **objeto de uma classe específica** possa ser **tratado como um objeto de uma classe mais genérica**
- **Facilidade para o desenvolvimento de sistemas que sejam facilmente extensíveis**
 - Novas classes podem ser adicionadas ao sistema exigindo pouca ou nenhuma modificação nas partes gerais do sistema
 - Novas classes devem obrigatoriamente fazer parte de uma hierarquia de classes já existente no sistema
 - **Deve-se programar pensando somente nas classes mais genéricas**, não se preocupando com as classes mais específicas

Java of Empires



- Tudo que é possível fazer com um **Personagem** é possível fazer com um **Aldeao**, **Arqueiro** e **Cavaleiro**, mas não o contrário

```
Aldeao a = new Aldeao();
Personagem p = a;

a.mover(); // OK
p.mover(); // OK

a.coletarMadeira(); // OK
p.coletarMadeira(); // ERRO
```

Java of Empires

- O exército pode ter até 300 personagens
- É desejado mover todos os personagens do exército

```
Aldeao    vetA[] = new Aldeao[100];
Arqueiro  vetB[] = new Arqueiro[100];
Cavaleiro vetC[] = new Cavaleiro[100];

//omitindo a criação dos objetos

// invocando o método mover de cada objeto
for(int i = 0; i < 100; i++){
    vetA[i].mover();
    vetB[i].mover();
    vetC[i].mover();
}
```

- Essa abordagem não é recomendada, pois limita em 100 o número de personagens de cada tipo
- Se um novo tipo de personagem for criado, será necessário modificar o código dentro do laço de repetição

Java of Empires

- Com o **polimorfismo** é possível incluir novos personagens no jogo sem que seja preciso modificar boa parte do código
 - Basta adicionar um novo tipo de personagem e incluí-lo no vetor de personagens
 - O método `mover` será chamado de acordo com o tipo de personagem
 - Programme para o mais genérico e não para o mais específico

```
// O vetor da superclasse pode armazenar objetos das suas subclasses
Personagem vetP[] = new Personagem[300];

vetP[0] = new Aldeao();
vetP[1] = new Arqueiro();
vetP[2] = new Cavaleiro();
vetP[3] = new Soldado();

for(int i=0; i < 4; i++){
    // Polimorfismo. Em tempo de execução é determinado qual implementação será invocada
    vetP[i].mover();
}
```

Exercícios

Herança múltipla com Interfaces

- Faça um diagrama UML com classes e interfaces que permitam representar animais em um jogo
 - Arara, Avestruz, Baleia, Cachorro, Gato, Leão, Lobo, Macaco, Morcego, Pinguim, Ornitorrinco
- Todo animal possui um nome e no mínimo 2 habilidades
 - Exemplo: cachorro late e corre, arara voa e fala, leão ruge e corre
 - Método `habilidade` deve retornar uma `String` com a habilidade do animal
 - Método `toString` deve retornar o nome do animal e de qual classe ele é. Exemplo: "Totó é um cachorro"
- Implemente as classes em Java e faça um aplicativo Java para instanciar alguns objetos dessas classes em um único vetor e chamar suas habilidades usando polimorfismo

Gerar folha de pagamento de uma empresa

- Existem 4 carreiras na empresa
 - **mensal fixo** – valor fixo por mês independente do número de horas que trabalhou em um mês
 - **horista** – valor adicional pago por hora extra trabalhada além das 40 horas semanais. O valor da hora extra é acordado com cada funcionário
 - **comissionado** – salário calculado somente sobre o percentual das vendas que efetivou. O percentual das vendas é um valor acordado com cada funcionário
 - **comissionado efetivo** – valor fixo por mês, independente do número de horas que trabalhou em um mês, mais o percentual das vendas que efetivou
 - O percentual das vendas é um valor acordado com cada funcionário

Gerar folha de pagamento de uma empresa

- Faça uma modelagem para representar os funcionários dessa empresa
- Faça uma rotina que permita gerar a folha de pagamento da empresa
 - A folha de pagamento deve conter o nome do funcionário e o valor a ser pago
- Faça uma rotina que permita aumentar em 10% o salário base de todos os funcionários da carreira **comissionado efetivo**